

Operating System Structures

Placide Rwego Mucyo

Topic 2

Operating System Structures

- **Operating System Structure** refers to the way the components of an operating system are organized and how they communicate with one another.
- A good OS structure makes the system **efficient, reliable, secure, maintainable, and easier to develop.**
- The major operating system structures include **simple structure, monolithic structure, layered structure, microkernel structure, modular structure, hybrid structure, and virtual-machine structure.**

Operating System Structures

1. Simple Structure

In a simple operating system, there is little separation between the different components of the system.

Operating-system functions may be combined into a relatively small and loosely organized program.

Characteristics

- Simple organization.
- Limited separation between components.
- Components may directly interact with hardware.
- Easy to implement for small systems.

Example: Early operating systems such as MS-DOS used a relatively simple structure.

2. Monolithic Structure

A monolithic operating system places most operating-system services inside a **single large kernel**.

The kernel may contain:

Process management

Memory management

File-system management

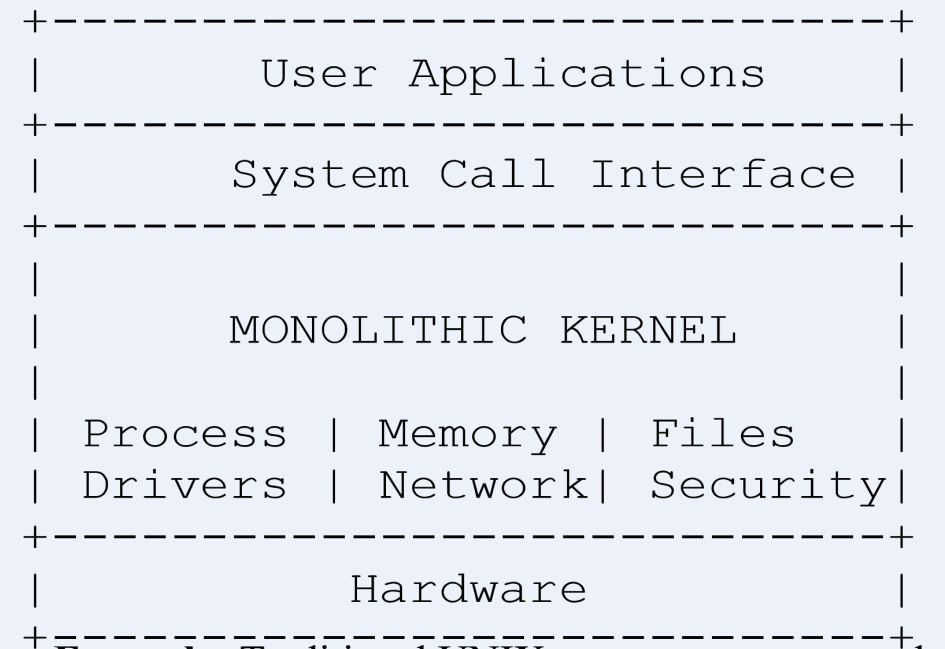
Device drivers

Networking

Security mechanisms

All these services operate within the kernel's privileged address space.

Basic Structure



Example: Traditional UNIX systems are commonly associated with monolithic kernel designs; **Linux** is monolithic but also supports modular kernel components.

3. Layered Structure

In a layered operating system, the OS is divided into several layers.

Each layer provides services to the layer above it and uses services from the layer below it.

Example

```
+-----+
| Layer 5: User Applications |
+-----+
| Layer 4: User Interface   |
+-----+
| Layer 3: File Management  |
+-----+
| Layer 2: Process Management|
+-----+
| Layer 1: Memory Management|
+-----+
| Layer 0: Hardware         |
+-----+
```

4. Microkernel Structure

A microkernel keeps only the most essential functions inside the kernel. Other operating-system services run outside the kernel, often as separate processes.

The microkernel may provide:

- Basic process management**

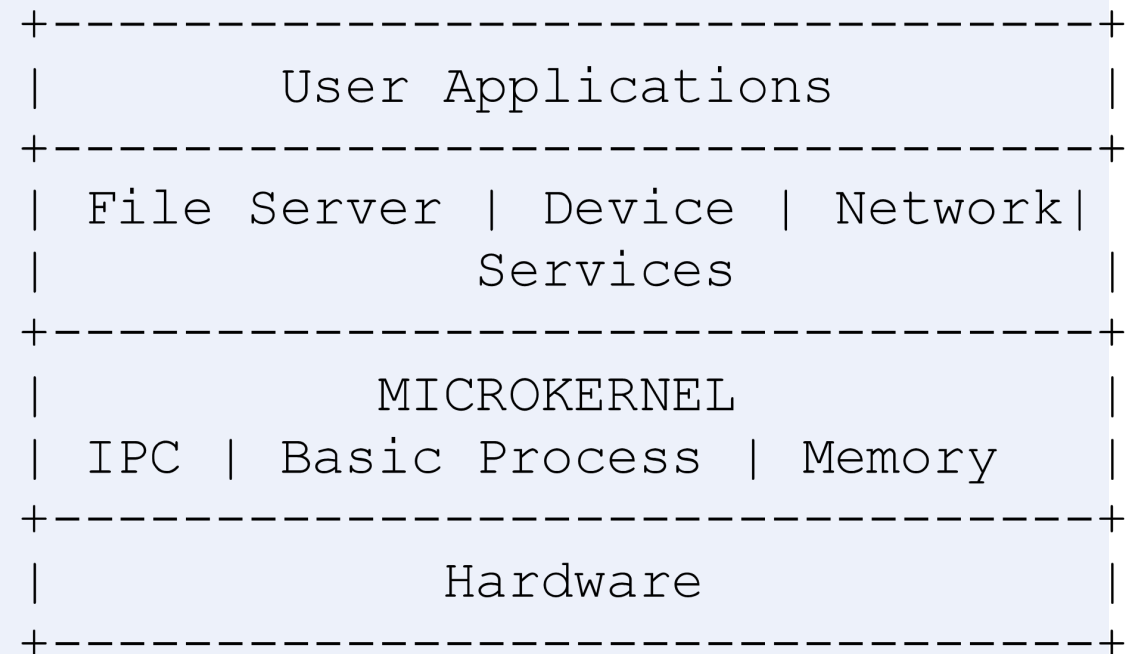
- Inter-process communication (IPC)**

- Basic memory management**

- Low-level hardware management**

Other services, such as file systems and device drivers, can operate in user space.

Structure



5. Modular Structure

A modular operating system uses a core kernel together with separate modules that can be loaded or removed when required.

For example, a module may provide:

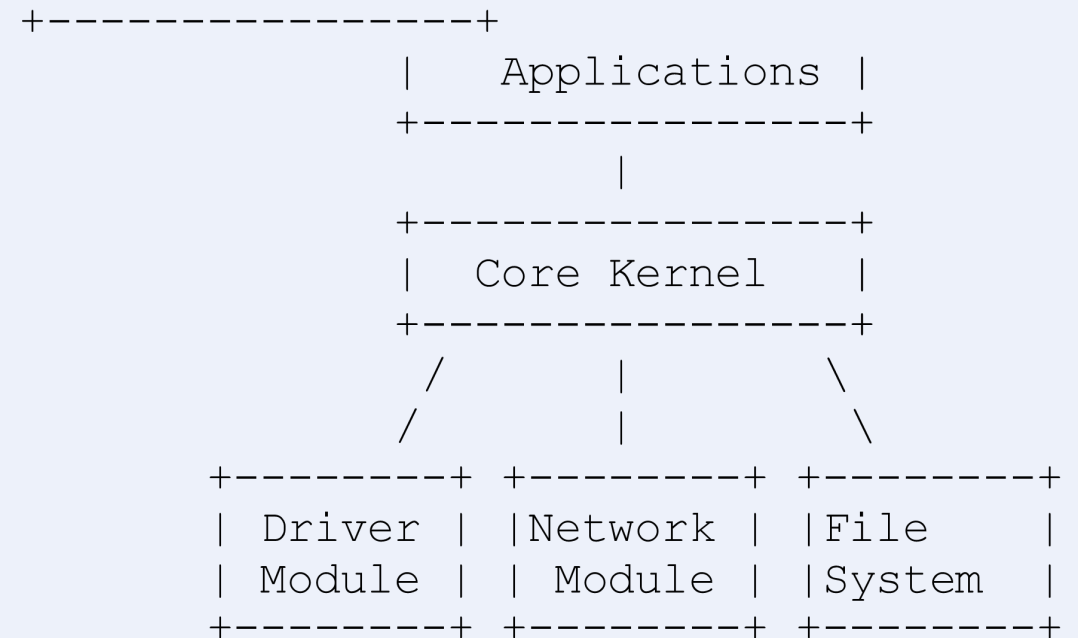
A device driver

A file system

Network functionality

Hardware support

Structure



Example: Linux supports loadable kernel modules.

6. Hybrid Structure

A hybrid operating system combines characteristics of different operating-system structures, commonly combining monolithic and microkernel concepts.

It attempts to achieve:

High performance

Modularity

Reliability

Flexibility

Examples

Windows uses a hybrid kernel architecture.

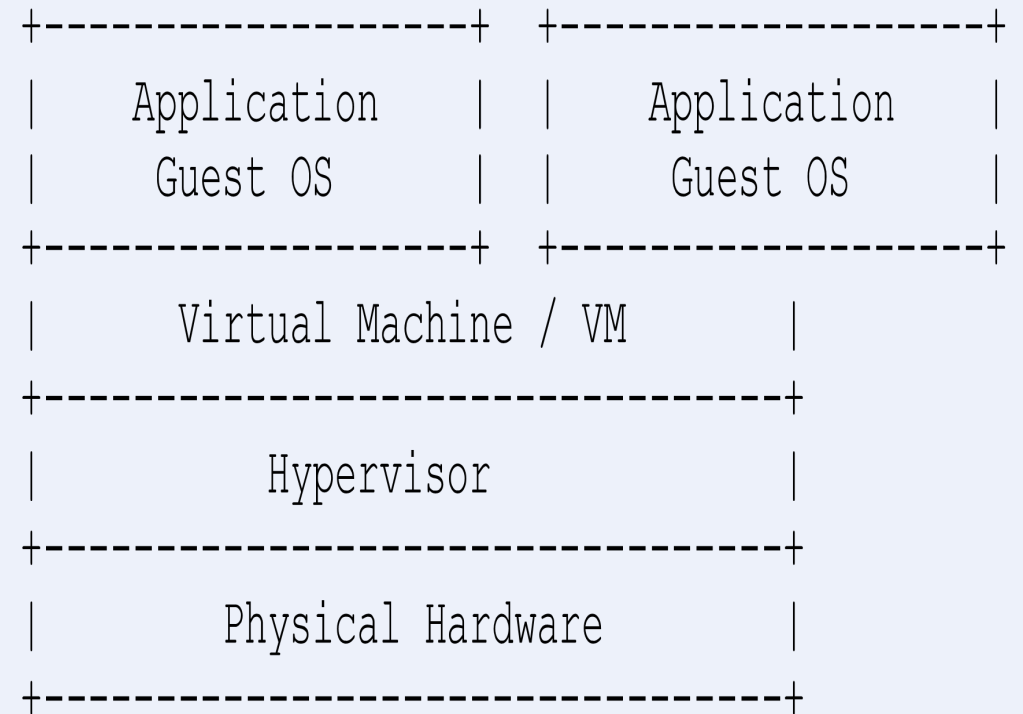
Apple's XNU kernel combines Mach-based components with BSD and other components.

7. Virtual-Machine Structure

A virtual machine (VM) creates an environment that behaves like an independent computer.

A hypervisor manages the underlying physical hardware and allows multiple virtual machines to run on the same physical computer.

Structure

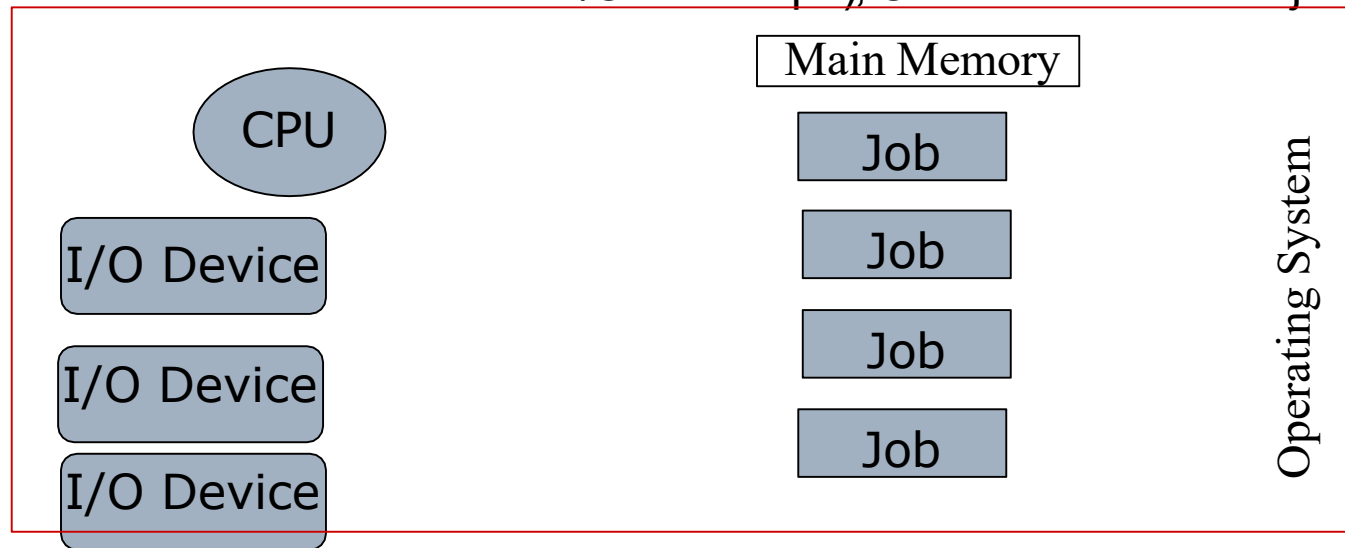


Comparison of Operating System Structures

Structure	Main Idea	Major Advantage	Major Disadvantage
Simple	Minimal organization	Easy to implement	Difficult to maintain
Monolithic	Most services in kernel	High performance	Complex kernel
Layered	OS divided into layers	Easy to understand	Layer design can be difficult
Microkernel	Minimal kernel, services outside	Reliability and security	Communication overhead
Modular	Core kernel + modules	Flexible	Module management complexity
Hybrid	Combines multiple approaches	Performance and flexibility	Complex design
Virtual Machine	Virtual computers over hardware	Isolation and resource sharing	Virtualization overhead

Operating System Structure

- ❑ Multiprogramming needed for efficiency
 - Single user cannot keep CPU and I/O devices busy at all times
 - Multiprogramming organizes jobs(Code and data) so CPU always has one to execute
 - A Subset of total jobs in system is kept in memory
 - One job selected and run via job scheduling
 - ❑ OS selects which job
 - ❑ When it has to wait for I/O for example), OS switches to another job



Operating System Structure

- ❑ Timesharing (**Multitasking**) is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating interactive computing
 - Response time should be <1 second
 - Each user has at least one program executing in memory(Processes)
 - If several jobs ready to run at the same time **CPU Scheduling** is performed
 - If processes do not fit in memory, **swapping** moves them in and out to run
 - **Virtual memory** allows execution of processes not completely in memory

Key Concepts to Remember

- In summary, operating-system structure determines how OS components are organized, isolated, and communicate.
- The choice of structure involves trade-offs among **performance, security, reliability, flexibility, and maintainability.**

